

1. Creating Array(ipynb)

NumPy Basics for Numerical Computing

NumPy is the core library for numerical computing in Python and the foundation on which Pandas is built. It provides fast, efficient tools for creating, storing, and manipulating numeric data, making it essential for data science workflows.

NumPy arrays are created using `np.array()` and can be one-dimensional or multi-dimensional, depending on whether the input is a list or a list of lists. Array properties such as `ndim`, `shape`, and `dtype` describe the number of dimensions, size, and data type of the array.

NumPy enforces homogeneous data types within arrays and automatically upcasts values when needed, such as converting integers to floats. Arrays can be initialized with placeholder values using functions like `zeros`, `ones`, and `random.rand`, which are commonly used for testing and examples.

Sequences of values can be generated using `arange`, which specifies start, end (exclusive), and step size, or `linspace`, which generates a fixed number of evenly spaced values between two inclusive endpoints.

These array creation and inspection tools form the basis for understanding more advanced numerical and data analysis

libraries in Python.

2. Manipulating Array(ipynb)

NumPy Array Operations and Matrix Concepts

NumPy arrays support fast element-wise arithmetic operations such as addition, subtraction, multiplication, exponentiation, and comparisons. Applying operators to arrays works on each element individually, which makes data transformation concise and efficient.

NumPy is useful for real-world transformations, such as converting temperature values from Fahrenheit to Celsius using a simple formula applied to an entire array. Boolean arrays are created by applying logical conditions to arrays, returning True or False for each element. These are commonly used for filtering and conditional analysis.

Beyond element-wise operations, NumPy also supports matrix-level operations. The `*` operator performs element-wise multiplication, while the `@` operator performs matrix (dot) multiplication. Understanding this distinction is important when working with numerical and scientific data. Array shapes can be inspected using the `shape` attribute to ensure matrix operations are valid.

NumPy automatically upcasts data types when combining arrays of different types, such as integers and floats, resulting in the more general type. Arrays include aggregation functions like `sum`, `max`, `min`, and `mean`, which can be applied to entire arrays or along specific dimensions after reshaping.

These operations highlight why NumPy is the backbone of scientific and

data-centric computing in Python.

3. From Arrays to Images and Back(ipynb)

NumPy Arrays, Images, and Data Abstraction

Two-dimensional NumPy arrays can be viewed not only as rows and columns but also as large, ordered sequences of values. This concept is fundamental to how images are stored and manipulated in computing systems.

Images can be loaded using the Python Imaging Library (PIL) and converted into NumPy arrays. A grayscale image becomes a 2D array where each element represents a pixel intensity, typically stored as unsigned 8-bit integers (uint8) ranging from 0 (black) to 255 (white).

NumPy enables efficient image manipulation through element-wise operations. For example, image inversion can be performed by subtracting pixel values from 255 and adjusting the data type. After modification, arrays can be converted back into image objects and displayed.

Reshaping an array changes how the data is interpreted without altering the underlying values. Altering an image's shape while preserving the total number of elements changes its visual appearance, demonstrating that shape is an abstraction layered on top of raw data.

This example highlights a core idea in data science: data is fundamentally numeric, and meaning comes from the abstractions we build on top of it. A data scientist's role is to understand data representations and transform them into forms that support insight and interpretation.

4. Indexing, Slicing, and Iterating(ipynb)

NumPy Indexing, Slicing, and Boolean Selection

Indexing, slicing, and iteration are core NumPy skills for selecting, updating, and analyzing data. In one-dimensional arrays, integer indexing works like Python lists, using zero-based positions to access elements.

For multi-dimensional arrays, elements are accessed using row and column indices. Multiple specific elements can be selected using index arrays or paired index lists, which allows extraction of values into a new one-dimensional array.

Boolean indexing enables selection based on conditions. Applying a condition to an array produces a Boolean mask, which can then be used to filter values. This broadcasting and masking behavior is fundamental to NumPy and heavily used in Pandas.

Slicing creates subarrays using the colon syntax. In multi-dimensional arrays, the first slice selects rows and the second selects columns. Slicing returns a view into the original array, not a copy, meaning changes to the slice also modify the original data. This pass-by-reference behavior is important to understand to avoid unintended data changes.

These indexing and slicing techniques form the foundation for efficient data manipulation in Python data science workflows.

5. Trying NumPy with Datasets

Applying NumPy to Real Datasets (Explained with Examples)

NumPy is commonly used to load, inspect, and analyze structured numerical data such as CSV files. It allows fast selection, transformation, and summarization of data using array operations.

1. Loading Data with `genfromtxt`

NumPy loads CSV data using `np.genfromtxt`.

Example:

```
import numpy as np
```

```
wines = np.genfromtxt(  
    'winequality-red.csv',  
    delimiter=';',  
    skip_header=1  
)
```

- `delimiter` specifies how values are separated
- `skip_header=1` skips the column names
- NumPy infers numeric data types automatically

2. Selecting Rows and Columns

For 2D arrays:

- First index = rows
- Second index = columns

Example: Select first column (fixed acidity):

```
wines[:, 0]
```

Returns a **1D array** of values.

Preserve column shape:

```
wines[:, 0:1]
```

Returns a **2D column array**.

Same data, different shape. Shape is an abstraction.

3. Selecting Multiple Columns

Consecutive columns (0 to 2):

```
wines[:, 0:3]
```

Non-consecutive columns (alcohol and quality):

```
wines[:, [10, 11]]
```

4. Aggregation Functions

NumPy provides built-in summary statistics.

Example: Average wine quality

```
np.mean(wines[:, -1])
```

- -1 selects the last column
- Negative indexing counts from the end

5. Structured Arrays with Column Names

When column names are provided, NumPy creates structured arrays.

Example: Load admissions dataset

```
admissions = np.genfromtxt(  
    'Admission_Predict.csv',  
    delimiter=',',  
    names=True,  
    dtype=None  
)
```

Each row becomes a tuple, and columns are accessed by name.

Example: Access GPA values

```
admissions['CGPA'][:5]
```


6. Modifying Data in Place

Array operations directly modify data.

Example: Convert GPA from 10-point to 4-point scale

```
admissions['CGPA'] = admissions['CGPA'] / 10 * 4
```

7. Boolean Masking (Filtering Data)

Boolean expressions create masks used to filter rows.

Example: Students with research experience

```
mask = admissions['Research'] == 1
```

```
len(admissions[mask])
```

8. Conditional Analysis with Boolean Masks Example:

Compare GRE scores by admission chance

High chance (> 80%):

```
np.mean(admissions['GRE Score'][admissions['Chance of Admit'] > 0.8])
```

Low chance (< 40%):

```
np.mean(admissions['GRE Score'][admissions['Chance of Admit'] < 0.4])
```

This shows how conditions, filtering, and aggregation work together.

Key Takeaways

- NumPy arrays are fast, numeric, and shape-based
- Indexing and slicing control how data is viewed
- Boolean masking enables powerful conditional selection
- Aggregations answer real analytical questions
- Pandas builds directly on these NumPy concepts

